

AGENTIC AI

WEEK 3

Exploring CrewAI: Autonomous Agents for Complex Tasks

Document Type	Personal Study Reference — Publication Draft
Domain	Multi-Agent Systems / CrewAI Framework / LLM Orchestration
Week	Week 3 of 8 — CrewAI Deep Dive
Topics	CrewAI Overview, Agents/Tasks/Crews, Project Structure, YAML Config, Memory Types, Real-World Applications

TABLE OF CONTENTS

SECTION 1 — THEORETICAL FOUNDATIONS

- 1.1 What is CrewAI? — Philosophy, Design Goals & Architecture
- 1.2 The CrewAI Product Ecosystem
- 1.3 Core Concepts: Agents, Tasks, and Crews
- 1.4 Crew Execution Processes: Sequential vs. Hierarchical
- 1.5 LiteLLM — The Universal LLM Interface
- 1.6 Creating & Structuring a CrewAI Project
- 1.7 YAML-Based Configuration: Agents & Tasks
- 1.8 Memory Management in CrewAI
- 1.9 Tools & External Integrations (Serper.dev)
- 1.10 Real-World Application: Stock Picker (Sequential Crew)
- 1.11 Real-World Application: Engineering Team (Hierarchical Crew)

SECTION 2 — PRACTICAL IMPLEMENTATION

- 2.1 Installation & Environment Setup
- 2.2 Creating a New CrewAI Project via CLI
- 2.3 Defining Agents in YAML (agents.yaml)
- 2.4 Defining Tasks in YAML (tasks.yaml)
- 2.5 Implementing the Crew in crew.py
- 2.6 Running the Crew via main.py
- 2.7 Enabling Memory in a Crew
- 2.8 Complete Project: Stock Picker Crew — End-to-End

SECTION 1

Theoretical Foundations

This section provides a comprehensive theoretical treatment of the CrewAI framework — from its foundational design philosophy through its core abstractions, project structure conventions, memory architecture, and representative real-world application patterns. The concepts presented herein form the intellectual basis for the practical implementations demonstrated in Section 2.

1.1 What is CrewAI? — Philosophy, Design Goals & Architecture

A lean, independent Python framework for orchestrating high-performance multi-agent AI systems

CrewAI is an open-source Python framework designed specifically for the orchestration of collaborative, autonomous AI agents. A defining characteristic that distinguishes CrewAI from many contemporaneous frameworks is that it is built entirely from scratch — it does not depend upon LangChain, LlamaIndex, or any other pre-existing agent framework. This architectural independence makes CrewAI exceptionally lightweight, performant, and free from the dependency bloat and abstraction limitations that can arise when building atop another framework's assumptions.

The central design philosophy of CrewAI is the **crew metaphor**: the recognition that the most effective way to solve complex, multi-faceted problems is to assemble a team of specialists, each with a clearly defined role and area of responsibility, working in concert toward a shared objective. This metaphor is not merely illustrative — it is the literal architectural pattern around which the entire framework is constructed.

CrewAI is expressly designed for use cases requiring **auditable outcomes and precise control over execution**. Unlike purely reactive agentic systems where the path from input to output is entirely emergent and unpredictable, CrewAI provides mechanisms for structuring, monitoring, and governing the execution of multi-agent pipelines — making it suitable for production deployment in regulated or high-stakes domains.

Two Primary Paradigms

CrewAI offers two complementary approaches to multi-agent orchestration:

- **CrewAI Crews & Autonomous Solutions:** Teams of AI agents, each assigned a distinct role and goal, that collaborate autonomously to solve open-ended problems, engage in creative work, or conduct exploratory research. The crew collectively determines how to approach a task, with agents delegating to and consulting one another as appropriate.

- **CrewAI Flows:** A structured automation paradigm in which complex pipelines are decomposed into discrete, precisely defined workflow steps with deterministic execution paths. Flows are appropriate when the sequence of operations must be controlled, predictable, and auditable — analogous to a business process automation rather than an open-ended collaborative exploration.

Design Trade-off: Autonomy vs. Control

CrewAI Crews lean toward agent autonomy — the agents themselves decide how to divide and approach work. CrewAI Flows lean toward developer control — the developer prescribes the exact sequence of operations. The choice between these paradigms is a fundamental design decision that depends on the predictability requirements and auditability constraints of the target use case.

1.2 The CrewAI Product Ecosystem

From open-source framework to enterprise platform

CrewAI's commercial strategy encompasses three distinct product tiers, each targeting a different user segment and level of technical sophistication. Understanding this product landscape is important context for practitioners who may be evaluating CrewAI for organizational deployment.

Product	Target User	Key Capability	Use Case
CrewAI Open-Source Framework	Developers / Researchers	Full programmatic control; Python API; CLI tooling; complete customisability	Custom multi-agent application development
CrewAI Enterprise (Crew Enterprise)	Enterprise teams	Managed deployment, execution monitoring, observability dashboards, role-based access	Production deployment of multi-agent pipelines at scale
GensAI UI Studio 7	Non-technical users / Citizen developers	Visual, no-code / low-code drag-and-drop interface for constructing agent workflows	Rapid prototyping; business user-accessible agent building

For the purposes of this course, all practical work utilises the **CrewAI Open-Source Framework**, which is freely available via PyPI and provides the fullest degree of control and transparency — essential properties for learning purposes.

1.3 Core Concepts: Agents, Tasks, and Crews

The fundamental primitives of the CrewAI abstraction model

1.3.1 The Agent

An **Agent** is the fundamental autonomous unit of computation within CrewAI. Conceptually, an Agent models a skilled professional with a specific area of expertise, a defined objective, and the tools required to accomplish their work. Each Agent is instantiated with the following defining attributes:

Attribute	Type	Description
role	string	Defines the agent's professional identity and domain of expertise (e.g., 'Senior Financial Analyst', 'Python Backend Engineer'). This shapes the LLM's persona and influences how it approaches tasks.
goal	string	A clear, specific statement of what this agent is trying to achieve. The goal provides the agent with motivational direction and serves as a reference point for evaluating whether its actions are on-track.
backstory	string	A narrative description providing contextual depth to the agent's persona. The backstory allows the LLM to adopt a coherent professional identity, improving the relevance and quality of its outputs.
llm	string / LLM object	The specific language model powering this agent's reasoning (e.g., 'gemini-2.0-flash', 'gpt-4o-mini'). Different agents within the same crew may use different LLMs.

tools	list[Tool]	A list of external tool objects the agent is authorized to invoke (e.g., web search, code execution, file reading). Tools extend the agent's capabilities beyond pure text generation.
memory	bool	When True, the agent retains information across tasks within the current execution, enabling continuity and context accumulation.
verbose	bool	When True, the agent logs its reasoning process step by step, providing observability into its thought process — invaluable for debugging.
max_iter	int	Maximum number of iterations the agent will attempt before returning its best answer. Prevents runaway execution.

Agents may be defined either programmatically in Python code or declaratively in a YAML configuration file. The YAML approach is strongly preferred in production contexts as it cleanly separates agent configuration from application logic, facilitating easier modification and version control.

1.3.2 The Task

A **Task** represents a specific, well-scoped unit of work to be performed by a designated Agent. Tasks are the operational building blocks of a crew's workflow — they translate high-level objectives into concrete, actionable work items. Each Task is defined with the following attributes:

- **description:** A detailed natural-language description of exactly what the agent is expected to do. The quality and specificity of this description is one of the most significant factors influencing output quality.
- **expected_output:** An explicit statement of what a successful completion of this task looks like. This may describe format (e.g., 'a JSON object with keys X, Y, Z'), length (e.g., 'a 500-word report'), or content requirements.
- **agent:** A reference to the Agent instance responsible for executing this task. In a sequential crew, tasks are executed in the order they are defined. In a hierarchical crew, task assignment is managed by the manager agent.

- **context:** An optional list of other Task objects whose outputs should be provided as input context to this task. Enables the output of one task to inform the work of subsequent tasks.
- **output_file:** An optional file path; if specified, the task's output is automatically written to disk at that location upon completion.

1.3.3 The Crew

A **Crew** is the top-level orchestration entity that binds together a collection of Agents and Tasks into a coherent, executable unit. The Crew is responsible for managing the execution lifecycle: initiating task execution, routing task outputs between agents, managing shared memory, and producing a final consolidated result.

The Crew also defines the overall execution **process** — the strategy by which tasks are assigned and executed — and manages cross-cutting concerns such as memory, verbose logging, and the maximum number of requests per minute to prevent API rate limiting.

1.4 Crew Execution Processes: Sequential vs. Hierarchical

The two fundamental strategies for task orchestration within a Crew

CrewAI provides two distinct execution process modes that govern how tasks within a crew are assigned, ordered, and executed. The selection of the appropriate process is a critical architectural decision that significantly impacts system behaviour, complexity, and flexibility.

Sequential Process

In the **Sequential** process, tasks are executed in the exact order in which they are defined in the crew's task list. Each task receives, as its input context, the outputs of all preceding tasks. This creates a linear pipeline in which information flows unidirectionally from the first task to the last.

The sequential process is appropriate for workflows with a clear, natural ordering — where each step logically depends on the completion of the previous step and the final output is the result of the last task. It is simpler to reason about, easier to debug, and more predictable than the hierarchical alternative.

Example: In a content creation pipeline, Task 1 (Research Agent) gathers information, Task 2 (Writing Agent) drafts content using the research, and Task 3 (Editor Agent) refines the draft. Each step clearly depends on the prior step's output.

Hierarchical Process

In the **Hierarchical** process, a designated *Manager Agent* (often a more capable LLM) takes responsibility for dynamically assigning tasks to worker agents based on the nature of each task, the capabilities of available agents, and the evolving state of the overall objective. Worker agents report their results back to the manager, who synthesizes these results and determines next steps.

The hierarchical process introduces significantly greater flexibility and intelligence into the task execution strategy — the manager can adapt task assignment in response to intermediate results, handle unexpected situations, and optimally utilize the specialized capabilities of different agents. However, it is more complex, less predictable, and inherently more expensive in terms of LLM API calls.

Dimension	Sequential	Hierarchical
Task Assignment	Predefined order, fixed at design time	Dynamic assignment by manager agent at runtime
Flexibility	Low — fixed pipeline	High — adaptive task routing
Predictability	High — fully auditable path	Lower — manager's decisions emerge at runtime
Complexity	Low — simple to implement and debug	High — requires capable manager LLM
API Cost	Lower — tasks run once in sequence	Higher — manager adds additional LLM calls
Best For	Clearly ordered, dependent workflows	Complex tasks with dynamic sub-task needs

1.5 LiteLLM — The Universal LLM Interface

How CrewAI achieves provider-agnostic model integration

Under the hood, CrewAI delegates all LLM communication to **LiteLLM** — an open-source library that provides a single, unified Python interface for over 100 different LLM providers and models. LiteLLM translates CrewAI's standardised API calls into the specific request format required by each provider, handling authentication, request formatting, response parsing, and error handling transparently.

This design decision has profound practical implications: by specifying a model string in an Agent's `llm` parameter, a developer can seamlessly switch between OpenAI's GPT-4o, Anthropic's Claude, Google's Gemini, Meta's LLaMA (via Groq or Ollama), Mistral, and dozens of other models — without changing any other code. Different agents within the same crew may use different models, allowing cost-performance optimization at the agent level.

LiteLLM Model String Format

Models are specified using LiteLLM's provider/model string convention. Examples: 'openai/gpt-4o-mini', 'anthropic/claude-3-5-sonnet-20241022', 'gemini/gemini-2.0-flash', 'groq/llama3-70b-8192', 'ollama/llama3' (local). If only the model name is given (e.g., 'gemini-2.0-flash'), LiteLLM infers the provider from the model name.

1.6 Creating & Structuring a CrewAI Project

Convention-over-configuration project scaffolding via the CrewAI CLI

CrewAI provides a command-line interface (CLI) that scaffolds a complete, correctly structured project directory with a single command. This follows the *convention over configuration* principle: by generating a standardised project layout, the CLI eliminates boilerplate decisions and ensures that all CrewAI projects share a consistent, familiar structure.

The canonical project creation command is:

In [shell]:

BASH

```
crewai create crew my_project_name
```

The generated project structure is as follows:

```
my_project_name/
├── src/
│   ├── my_project_name/
│   ├── __init__.py
│   ├── main.py # Entry point: configures and launches the crew
│   ├── crew.py # Core file: defines Agents, Tasks, and the Crew
│   └── config/
│       ├── agents.yaml # Declarative agent definitions (role, goal, backstory, llm)
│       └── tasks.yaml # Declarative task definitions (description, expected_output)
├── tests/
├── .env # API keys (never commit to version control)
├── pyproject.toml # Python packaging metadata and dependencies
└── README.md
```

Purpose of Each File

- **agents.yaml:** A YAML configuration file containing the declarative definitions of all agents — their roles, goals, backstories, and LLM assignments. Keeping agent configuration in YAML separates

concerns cleanly and allows non-developers to modify agent personas without touching Python code.

- **tasks.yaml:** A YAML configuration file defining all tasks — their descriptions and expected outputs. Task definitions in YAML serve as the 'specification' layer, while `crew.py` serves as the 'wiring' layer that connects tasks to agents.
- **crew.py:** The Python module where agents and tasks are instantiated (using the YAML configs as data sources), assembled into a Crew object, and assigned a process. This file contains the core orchestration logic.
- **main.py:** The application entry point. This file invokes the crew's kickoff method with the initial inputs, handles results, and manages any additional application-level configuration (e.g., setting environment variables, configuring logging).

API Key Configuration

External service API keys (OpenAI, Gemini, Groq, Serper, etc.) are stored in the `.env` file at the project root and loaded at runtime. For web search capabilities, Serper.dev provides a generous free tier with a simple API key — obtain one at serper.dev, add `SERPER_API_KEY=your_key` to your `.env` file, and CrewAI's built-in SerperDevTool handles the rest.

1.7 YAML-Based Configuration: Agents & Tasks

Declarative agent and task definitions — separating configuration from logic

The YAML configuration files are a cornerstone of CrewAI's design philosophy. By externalising agent and task definitions into structured configuration files, CrewAI achieves a clean separation between the *what* (what roles exist, what tasks must be done) and the *how* (the Python code that instantiates and orchestrates them). This separation yields several important benefits: configuration changes do not require code modifications; different configurations can be maintained for different environments; and non-technical stakeholders can review or modify agent personas without engaging with application code.

agents.yaml — Structure & Best Practices

Each top-level key in `agents.yaml` is the logical identifier of an agent (used to reference it in `crew.py`). Each agent definition is a mapping of attribute names to their values. The four most critical attributes are `role`, `goal`, `backstory`, and `llm`. These directly shape the agent's LLM prompt and therefore its behaviour.

tasks.yaml — Structure & Best Practices

Each top-level key in `tasks.yaml` is the logical identifier of a task. The two mandatory attributes are `description` — which should be detailed, specific, and unambiguous — and `expected_output`, which serves as the quality specification for the task's result. The agent assignment is handled in

crew.py, not in the YAML, maintaining the separation of concerns.

1.8 Memory Management in CrewAI

Three distinct memory types for context retention and knowledge accumulation

One of CrewAI's most powerful capabilities for building intelligent, contextually-aware agents is its multi-layered memory architecture. By default, LLM interactions are stateless — each call is independent with no inherent recall of prior interactions. CrewAI's memory system addresses this limitation by providing three distinct types of memory that serve different temporal and semantic purposes.

1.8.1 Short-Term Memory (Temporary Memory)

Short-term memory stores **recent information and intermediate outcomes** produced during the *current* crew execution. It functions as a working memory — a scratchpad that allows agents to access results produced by prior tasks within the same run without needing to re-derive them.

Technically, short-term memory is implemented using an in-memory vector store that enables semantic search over recent task outputs. When an agent needs information relevant to its current task, it can retrieve semantically similar content from short-term memory even if the exact content was not explicitly passed as a task context parameter. This significantly reduces the need for verbose, explicit inter-task data passing.

1.8.2 Long-Term Memory

Long-term memory provides **persistence across multiple crew executions**. Unlike short-term memory, which is discarded when a crew execution completes, long-term memory is written to a persistent store (typically a local SQLite database or a configurable vector database) and is available to agents in future runs.

This capability enables CrewAI agents to genuinely accumulate knowledge over time — learning from past experiences, building domain expertise incrementally, and improving decision-making quality with each successive execution. Long-term memory is particularly valuable for agents performing recurring research, monitoring, or advisory tasks where historical context enhances current performance.

1.8.3 Entity Memory

Entity memory is a specialised memory type focused on **structured information about named entities** — people, organisations, products, locations, and concepts — encountered during task execution. Rather than storing raw text, entity memory extracts and organises entity-centric facts into a structured knowledge graph, enabling agents to build a cumulative, interconnected understanding of the entities relevant to their domain.

Entity memory is particularly powerful in conjunction with Retrieval-Augmented Generation (RAG): the agent can retrieve structured entity information as additional context for its reasoning, grounding its responses in accumulated knowledge about specific entities rather than relying solely on its LLM pre-training. This is especially valuable in knowledge-intensive domains such as financial research, medical analysis, and competitive intelligence.

Memory Type	Scope	Storage	Primary Use Case
Short-Term (Temporary)	Current execution only	In-memory vector store	Sharing intermediate results between tasks within one run
Long-Term	Across multiple executions	Persistent vector/SQL database	Accumulating knowledge and expertise over repeated runs
Entity Memory	Current & across executions	Structured knowledge graph / vector store	Building contextual understanding of specific named entities

Enabling Memory

Memory is activated at the Crew level by setting `memory=True` in the Crew constructor within `crew.py`. When enabled, all three memory types are automatically activated. Individual memory components can be configured separately for more granular control.

1.9 Tools & External Integrations

Extending agent capabilities with real-world data access and actions

Tools are one of the most critical enablers of genuine agent utility. Without tools, an agent is limited to reasoning about information already present in its training data or explicitly provided in its prompt. Tools allow agents to actively *gather* new information, *perform* computations, *interact* with external systems, and *take actions* with real-world consequences.

CrewAI provides a rich built-in tool library and supports the creation of custom tools. Built-in tools include web search, website scraping, file reading/writing, code execution, GitHub integration, database connectors, and many others. Tools are assigned to individual agents — an agent may only use tools explicitly listed in its `tools` parameter.

Serper.dev — Web Search Integration

For web search capabilities, CrewAI integrates with **Serper.dev**, a Google Search API that provides real-time web search results via a simple REST API. Serper.dev offers a free tier with a sufficient quota for development and light production usage, making it the recommended starting point for search-enabled CrewAI agents.

Once a `SERPER_API_KEY` is set in the `.env` file, the `SerperDevTool` from CrewAI's tools module can be instantiated and assigned to any agent that requires web search access. The tool handles query construction, HTTP communication with the Serper API, response parsing, and result formatting — all transparently.

1.10 Real-World Application: Stock Picker (Sequential Crew)

A two-agent financial research pipeline demonstrating sequential task execution

The Stock Picker project illustrates the power of a sequential two-agent crew for structured financial research and recommendation generation. This application demonstrates how CrewAI can automate a research pipeline that would otherwise require significant manual effort from human analysts.

Agent Architecture

- **Financial Researcher Agent:** Responsible for gathering comprehensive, current information about specified companies, sectors, and macroeconomic conditions. This agent is equipped with the `SerperDevTool` for real-time web search and the `WebsiteScrapingTool` for extracting structured data from financial websites, news portals, and company investor relations pages.
- **Stock Analyst Agent:** Receives the research outputs from the Financial Researcher and applies quantitative and qualitative analytical frameworks to evaluate the data, identify investment opportunities, assess risk, and formulate stock recommendations with supporting rationale.

Task Workflow

The sequential execution proceeds as follows: the Financial Researcher's task executes first, producing a structured research report as its output. This report is automatically passed as context to the Stock Analyst's task, which processes the research data and generates a ranked list of stock recommendations with investment theses.

This architecture delivers several important properties: clear separation of concerns between data gathering and analytical reasoning; independent auditability of each stage's output; and the ability to swap either agent's model independently to optimise cost-performance trade-offs (e.g., using a cheaper, faster model for research gathering and a more capable model for analytical synthesis).

Why Sequential for Stock Picker?

The stock picker workflow has a clear, natural dependency chain: analysis cannot proceed without research data, and research data must be current. A sequential process perfectly models this dependency: Task 1 always completes before Task 2 begins, guaranteeing that the analyst always operates on the researcher's latest output.

1.11 Real-World Application: Engineering Team (Hierarchical Crew)

A four-agent software development team demonstrating hierarchical task management

The Engineering Team project demonstrates the hierarchical process in a software development context. This multi-agent system models a realistic engineering team structure with an Engineering Lead coordinating the work of three specialist engineers, each responsible for a distinct layer of the software stack.

Agent Architecture

- **Engineer Lead (Manager Agent):** The orchestrating agent responsible for decomposing incoming software development requests into component tasks, assigning those tasks to the appropriate specialist engineers based on their roles, reviewing submitted work, and synthesizing individual deliverables into a coherent final product.
- **Backend Engineer:** Specialises in server-side logic — API design, database schema, business logic implementation, and data processing. Produces Python/server-side code and documentation.
- **Frontend Engineer:** Specialises in user interface development — HTML/CSS/JavaScript, component architecture, and user experience considerations. Produces frontend code and UI specifications.
- **Test Engineer:** Specialises in quality assurance — writing unit tests, integration tests, and end-to-end tests; identifying edge cases; and producing test reports that validate the correctness of the backend and frontend implementations.

Why Hierarchical for an Engineering Team?

The hierarchical process is the correct choice here because the decomposition of a software project into specific sub-tasks is itself a complex, context-dependent decision that benefits from intelligent management. A simple sequential pipeline would fix the task order in advance, but the Engineer Lead can dynamically assess incoming requirements and determine whether, for example, the backend needs to be completed before the frontend can begin, or whether both can be worked on in parallel with the Lead coordinating the integration.

Furthermore, the hierarchical structure naturally supports code review and iterative refinement: the Lead can review an engineer's output, identify deficiencies, and re-assign revision tasks — behaviour

that would require complex conditional logic to implement in a sequential workflow.

SECTION 2

Practical Implementation

This section presents the complete, annotated practical implementation of a CrewAI project from initial installation through to a fully functional multi-agent crew. Each code cell and configuration file is presented with comprehensive explanations of its purpose, structure, and the effect of each parameter. All examples are designed for immediate execution within a properly configured Python virtual environment.

Prerequisites

Ensure that Python 3.10+ is installed, a virtual environment is activated, and a .env file containing GEMINI_API_KEY (or OPENAI_API_KEY) and SERPER_API_KEY has been created at the project root before proceeding.

2.1 Installation & Environment Setup

Installing CrewAI and its optional tool dependencies

Cell 1 — Installing CrewAI

In [1]:

PYTHON

```
# Install the core CrewAI framework
# 'crewai[tools]' installs the framework plus the optional tools package
# which includes SerperDevTool, WebsiteScrapeTool, FileReadTool, etc.
!pip install 'crewai[tools]'

# Verify the installation
!crewai --version
```

Explanation:

The crewai[tools] package specification uses Python's extras syntax to install not only the core crewai package but also the optional crewai-tools package in a single command. Without the [tools] extra, built-in tools such as SerperDevTool and WebsiteScrapeTool would not be available. The crewai --version command confirms the CLI is accessible, verifying that the installation completed successfully.

Expected Output:

Out [1]:


```
Successfully installed crewai-0.x.x crewai-tools-0.x.x ...  
crewai, version 0.x.x
```

2.2 Creating a New CrewAI Project via CLI

Using the crewai CLI to scaffold a complete project structure

Cell 2 — Scaffolding the Project

In [2]:

PYTHON

```
# Navigate to your desired parent directory first  
import os  
  
os.makedirs('projects', exist_ok=True)  
os.chdir('projects')  
  
# Use the CrewAI CLI to create a new crew project  
# This generates the complete directory structure and all boilerplate files  
!crewai create crew stock_picker
```

Explanation:

The `crewai create crew <project_name>` command invokes CrewAI's project scaffolding engine, which generates a complete, immediately runnable project directory containing all required files in the correct locations. The project name is used as both the directory name and the Python package name — it should therefore follow Python naming conventions: lowercase, underscores (no spaces or hyphens). After scaffolding, the developer's work consists entirely of filling in the configuration files and implementing the crew logic — not managing project structure.

Expected Output:

Out [2]:

```
Creating crew stock_picker...  
Creating directory: stock_picker/  
Creating directory: stock_picker/src/stock_picker/config/  
Creating file: stock_picker/src/stock_picker/config/agents.yaml  
Creating file: stock_picker/src/stock_picker/config/tasks.yaml  
Creating file: stock_picker/src/stock_picker/crew.py  
Creating file: stock_picker/src/stock_picker/main.py  
Creating file: stock_picker/pyproject.toml  
Crew stock_picker created successfully!
```

2.3 Defining Agents in YAML (agents.yaml)

Declaratively specifying agent personas, goals, and LLM assignments

The `agents.yaml` file is where each agent's identity and configuration is declared. The structure is a YAML mapping where each top-level key is the agent's logical identifier — a Python-friendly name used to reference the agent in `crew.py`.

```
src/stock_picker/config/agents.yaml
```

```
# ██████████  
# Agent: financial_researcher  
# Role: Gathers market and company data using web search  
# ██████████  
  
financial_researcher:  
role: >  
Senior Financial Research Analyst  
goal: >  
Conduct comprehensive research on companies, market trends,  
macroeconomic indicators, and recent financial news to provide  
data-driven insights that support investment decision-making.  
backstory: >  
You are a veteran financial research analyst with 15 years of  
experience covering global equity markets. You excel at  
synthesising diverse data sources – earnings reports, news  
sentiment, analyst ratings, and technical signals – into clear,  
structured research briefs that investment teams rely upon.  
llm: gemini/gemini-2.0-flash  
# ██████████  
# Agent: stock_analyst  
# Role: Evaluates research data and generates recommendations  
# ██████████  
  
stock_analyst:  
role: >  
Quantitative Stock Analyst  
goal: >  
Analyse the research data provided, evaluate each company's
```

```
investment merit using fundamental and quantitative frameworks,  
and produce a prioritised list of stock recommendations.  
backstory: >  
You are a CFA charterholder with deep expertise in both  
fundamental and quantitative equity analysis. You are known  
for rigorous, data-backed stock selection and clear, concise  
investment theses that help portfolio managers make decisions.  
llm: gemini/gemini-2.0-flash
```

Explanation — Field by Field:

- **Top-level key (e.g., `financial_researcher`):** This is the agent's identifier. It must be a valid Python identifier (no spaces, starts with a letter or underscore). This exact name is used in `crew.py` to bind the YAML definition to a Python Agent object via the `@agent` decorator.
- **role:** The `>` character begins a 'folded scalar' — a multi-line string that is folded into a single line when parsed. The role directly shapes the LLM's system prompt persona. Use specific, professional titles rather than generic terms for better performance.
- **goal:** A precise statement of the agent's objective. This appears in the agent's prompt and guides its decision-making. The more specific and measurable this statement, the more focused the agent's behaviour.
- **backstory:** Narrative context that provides the LLM with a rich persona to embody. Well-crafted backstories demonstrably improve output quality — they prime the model to respond as a domain expert rather than a generalist assistant.
- **llm:** The LiteLLM model string for this agent's underlying language model. Note the `gemini/` prefix, which is LiteLLM's provider prefix for Google's Gemini models.

2.4 Defining Tasks in YAML (tasks.yaml)

Specifying task descriptions and quality criteria declaratively

src/stock_picker/config/tasks.yaml

```
# ██████████  
# Task: research_task  
# Assigned to: financial_researcher agent  
# ██████████  
  
research_task:  
description: >  
  
Conduct thorough research on the following companies: {companies}.  
For each company, gather:
```

```
- Recent financial performance (revenue, EPS, guidance)
- Latest news headlines and sentiment analysis
- Analyst ratings and price targets
- Key risk factors and catalysts

Use web search to ensure all information is current.

expected_output: >

A structured research report in Markdown format containing
a dedicated section for each company with the four categories
of information requested above. Minimum 200 words per company.

# ████████████████████████████████████████████████████████████████

# Task: analysis_task

# Assigned to: stock_analyst agent

# ████████████████████████████████████████████████████████████████

analysis_task:

description: >

Using the research report provided, perform a rigorous
investment analysis of each company. Apply both fundamental
(P/E, P/B, ROE, debt ratios) and qualitative (management
quality, competitive moat, industry tailwinds) frameworks.
Produce a ranked recommendation list.

expected_output: >

A Markdown report containing: (1) an executive summary,
(2) individual company analysis sections, (3) a ranked
recommendation table (BUY / HOLD / SELL) with price targets
and investment thesis summaries, (4) key risk disclosures.
```

Explanation:

- **Task identifiers (research_task, analysis_task):** Used in crew.py to bind each YAML definition to a Python Task object. Must be valid Python identifiers.
- **description with {companies} placeholder:** CrewAI supports Jinja2-style template variables in task descriptions. The value of {companies} is substituted at runtime when the crew is kicked off with an inputs dictionary. This makes tasks reusable across different inputs without modifying the YAML.
- **expected_output:** Acts as the quality specification for the task. The more precisely this describes the desired format, length, and content, the more reliably the assigned agent will produce conformant output. Including format requirements (Markdown, JSON, etc.) is highly

recommended.

- **Agent assignment:** Note that `tasks.yaml` does NOT specify which agent performs each task — that binding is made in `crew.py`. This further reinforces the separation of concerns: task specifications are independent of agent assignments.

2.5 Implementing the Crew in crew.py

Wiring agents, tasks, and the crew together using decorators and the @CrewBase class

In [5]:

PYTHON

[illegible]

[illegible]

Explanation — Construct by Construct:

- **@CrewBase class decorator:** This decorator from `crewai.project` augments the class with automatic collection of all methods decorated with `@agent`, `@task`, and `@crew`. It also handles loading and parsing the YAML config files specified in `agents_config` and `tasks_config`, making them accessible as dictionaries via `self.agents_config` and `self.tasks_config`.
- **@agent decorator:** Marks a method as an agent factory. The method must return an `Agent` instance. The `config=self.agents_config['financial_researcher']` argument passes the parsed YAML attributes (role, goal, backstory, llm) directly to the `Agent` constructor. Additional Python-level attributes — such as `tools` and `verbose` — are specified here, supplementing the YAML configuration.

- **@task decorator:** Marks a method as a task factory. The `config` argument loads description and `expected_output` from YAML. The `agent` parameter binds the task to a specific agent. The `context` parameter explicitly declares that this task's prompt will include the output of the `research_task`. The `output_file` parameter instructs CrewAI to persist the task output to disk.
- **self.agents and self.tasks:** These are automatically populated by `@CrewBase` — they are lists of all instances returned by `@agent` and `@task` decorated methods, in definition order. Passing them to the Crew constructor registers all agents and tasks.
- **Process.sequential:** This enum value specifies that tasks will be executed in the order they are returned by `self.tasks` (i.e., the order they are defined in `crew.py`). Use `Process.hierarchical` for manager-directed execution.

2.6 Running the Crew via main.py

The application entry point — providing inputs and executing the crew

In [6]:

PYTHON

```
# src/stock_picker/main.py

import sys

from dotenv import load_dotenv

# Load API keys from .env before any CrewAI imports
# (CrewAI reads API keys from the environment at import time)
load_dotenv(override=True)

from stock_picker.crew import StockPickerCrew

def run():
    """Entry point for the Stock Picker crew."""
    # Define runtime inputs – substituted into {companies} placeholder in tasks.yaml
    inputs = {
        'companies': 'Apple (AAPL), Microsoft (MSFT), NVIDIA (NVDA)'
    }

    # Instantiate and kick off the crew
    result = StockPickerCrew().crew().kickoff(inputs=inputs)

    print('\n' + '='*60)
    print('CREW EXECUTION COMPLETE')
    print('='*60)
    print(result.raw) # Print the final task's raw output

if __name__ == '__main__':
```

```
run()
```

Explanation:

- **load_dotenv(override=True):** Called before any CrewAI imports because the framework reads API keys (GEMINI_API_KEY, SERPER_API_KEY, etc.) from environment variables at import time. Calling load_dotenv after importing CrewAI may result in missing API key errors.
- **inputs dictionary:** A Python dictionary mapping template variable names to their runtime values. The key 'companies' corresponds exactly to the {companies} placeholder in tasks.yaml. CrewAI performs string interpolation to substitute this value into the task description before the agent processes it.
- **StockPickerCrew().crew().kickoff(inputs=inputs):** This single line chains three operations: (1) instantiates the StockPickerCrew class; (2) calls the .crew() method to build and return the configured Crew object; (3) calls .kickoff() to begin crew execution. The call is synchronous and blocks until all tasks complete.
- **result.raw:** The CrewOutput object returned by kickoff() contains several attributes. .raw is the raw string output of the final task. .tasks_output is a list of TaskOutput objects, one per task, enabling access to intermediate results.

Expected Output (abbreviated):

Out [6]:

```
[2024-01-15 10:23:01][INFO]: Starting crew execution...

# Agent: Senior Financial Research Analyst
## Task: Conduct thorough research on the following companies: Apple ...

> Entering new chain...
> Calling tool: SerperDevTool
> Query: 'Apple AAPL latest earnings Q4 2024'
> Tool Result: [Search results returned...]
...
## Task Output:
# Financial Research Report
## Apple Inc. (AAPL)
### Recent Financial Performance ...

# Agent: Quantitative Stock Analyst
## Task: Using the research report provided, perform...
...

=====
CREW EXECUTION COMPLETE
```



```
=====
# Stock Investment Analysis Report
## Executive Summary ...
```

The verbose output reveals CrewAI's internal execution flow: each agent announces its task, logs its tool calls and intermediate reasoning steps, and produces its output. This transparency is invaluable for debugging and auditing the crew's behaviour. When `verbose=False`, only the final output is printed.

2.7 Enabling Memory in a Crew

Activating short-term, long-term, and entity memory with a single parameter

In [7]:

PYTHON

```
# src/stock_picker/crew.py – modified crew() method with memory enabled
@crew
def crew(self) -> Crew:
    return Crew(
        agents=self.agents,
        tasks=self.tasks,
        process=Process.sequential,
        memory=True, # <-- Enables ALL three memory types
        verbose=True,
        # Optional: configure individual memory components
        # embedder={
        #     'provider': 'google',
        #     'config': {'model': 'models/text-embedding-004'}
        # }
    )
```

Explanation:

Setting `memory=True` in the `Crew` constructor activates all three memory subsystems simultaneously. Internally, CrewAI initialises: a short-term in-memory vector store (cleared at the end of each execution); a long-term persistent store (defaulting to a local SQLite database file); and an entity memory store.

The optional `embedder` parameter allows the developer to specify which embedding model is used for the vector stores. By default, CrewAI uses OpenAI's embedding API. When working with Gemini models, it is recommended to configure the `embedder` explicitly to use Google's text embedding

model for consistency and to avoid cross-provider API key requirements.

Out [7]:

```
[INFO]: Short-term memory initialised.  
[INFO]: Long-term memory database: ./db/crew_memory.db  
[INFO]: Entity memory initialised.  
[INFO]: Memory system ready. Agents will retain context across tasks.
```

These log lines confirm that all three memory subsystems are active. In subsequent runs of the same crew, agents will be able to retrieve relevant information from the long-term store, enabling progressive improvement in response quality as the crew accumulates domain-specific knowledge.

2.8 Complete Project: Hierarchical Engineering Team Crew

A four-agent hierarchical crew demonstrating manager-directed task orchestration

This complete example demonstrates the hierarchical process with a four-agent engineering team. This is a more complex implementation that showcases the manager agent pattern and the use of memory to maintain project context across agent interactions.

src/eng_team/config/agents.yaml (excerpt)

```
engineer_lead:  
  role: >  
  Principal Software Engineer & Technical Lead  
  goal: >  
  Oversee the software project, decompose requirements into  
  clear engineering tasks, delegate to specialist engineers,  
  review deliverables, and ensure a coherent final product.  
  backstory: >  
  You are a seasoned technical lead with 12 years of full-stack  
  development experience. You excel at system design, code review,  
  and coordinating cross-functional engineering teams.  
  llm: openai/gpt-4o  
  
backend_engineer:  
  role: Senior Backend Engineer  
  goal: Design and implement robust, scalable server-side components.  
  backstory: >  
  Expert Python developer specialising in REST APIs, databases,
```

```
and microservices architecture.
llm: openai/gpt-4o-mini

frontend_engineer:
role: Senior Frontend Engineer
goal: Build intuitive, responsive user interfaces.
backstory: >
Specialist in React, TypeScript, and modern CSS. You focus
on accessibility, performance, and exceptional UX.
llm: openai/gpt-4o-mini

test_engineer:
role: QA Engineer
goal: Ensure code quality through comprehensive automated testing.
backstory: >
Quality-obsessed testing specialist with expertise in pytest,
Playwright, and testing best practices.
llm: openai/gpt-4o-mini
```

In [8]:**PYTHON**

```
# src/eng_team/crew.py – Hierarchical Engineering Team
from crewai import Agent, Task, Crew, Process
from crewai.project import CrewBase, agent, task, crew

@CrewBase
class EngineeringTeamCrew:
    """Hierarchical engineering team with GPT-4o as the technical lead."""

    agents_config = 'config/agents.yaml'
    tasks_config = 'config/tasks.yaml'

    @agent
    def engineer_lead(self) -> Agent:
        return Agent(
            config=self.agents_config['engineer_lead'],
            verbose=True,
            allow_delegation=True, # Critical: allows manager to delegate
        )

    @agent
```

```
def backend_engineer(self) -> Agent:
    return Agent(config=self.agents_config['backend_engineer'], verbose=True)

@agent
def frontend_engineer(self) -> Agent:
    return Agent(config=self.agents_config['frontend_engineer'], verbose=True)

@agent
def test_engineer(self) -> Agent:
    return Agent(config=self.agents_config['test_engineer'], verbose=True)

@task
def development_task(self) -> Task:
    return Task(
        config=self.tasks_config['development_task'],
        # No agent specified – manager assigns dynamically
        output_file='engineering_output.md',
    )

@crew
def crew(self) -> Crew:
    return Crew(
        agents=self.agents,
        tasks=self.tasks,
        process=Process.hierarchical, # Manager-directed execution
        manager_agent=self.engineer_lead(), # Designate the manager
        memory=True,
        verbose=True,
    )
```

Key Implementation Notes:

- **allow_delegation=True** on the manager agent: This parameter is critical for the hierarchical process. It permits the Engineer Lead to delegate sub-tasks to other agents in the crew. Without this flag, the manager agent cannot assign work to worker agents.
- **No agent specified in development_task:** In a hierarchical process, the manager agent dynamically determines which worker agent should handle each task (or sub-task) based on their roles and goals. Specifying a static agent assignment would undermine the hierarchical pattern's purpose.

- **process=Process.hierarchical with manager_agent:** The `manager_agent` parameter designates which agent acts as the orchestrator. This agent receives the overall objective, decomposes it, delegates, reviews, and synthesizes results.
- **Different LLMs for manager vs. workers:** The manager uses GPT-4o (more capable, more expensive) while workers use GPT-4o-mini (faster, cheaper). This cost-conscious architecture is a best practice — invest in capability where complex reasoning is required (management/planning) and optimize for cost where focused execution is sufficient.

Chapter Summary & Key Takeaways

1. CrewAI is an independent, lightweight Python framework for multi-agent orchestration that builds entirely from scratch — free from LangChain dependency — optimised for performance, auditability, and precise execution control.
2. The three core primitives — Agent (autonomous specialist), Task (unit of work), Crew (orchestrating team) — map cleanly to professional team metaphors, making system design intuitive and communicable to non-technical stakeholders.
3. YAML-based configuration (`agents.yaml` / `tasks.yaml`) separates agent persona and task specification from application logic in `crew.py`, enabling clean versioning, team collaboration, and non-technical modification of agent behaviour.
4. Sequential process is appropriate for clearly ordered, dependent workflows; Hierarchical process is appropriate for complex, dynamic tasks where a capable manager agent should direct and adapt task assignment at runtime.
5. LiteLLM's universal interface enables different agents within the same crew to use different LLMs — a powerful cost-performance optimisation strategy where capable (expensive) models are reserved for planning and management roles.
6. CrewAI's three-tier memory system — Short-Term (intra-run context), Long-Term (persistent cross-run knowledge), Entity Memory (structured entity knowledge graph) — enables agents to accumulate expertise and context awareness over time.
7. Tools are the mechanism by which agents access the real world: web search (SerperDevTool), website scraping, file I/O, code execution. Tool assignment is per-agent, enabling fine-grained capability control and the principle of least privilege.
8. The Stock Picker (sequential, two-agent) and Engineering Team (hierarchical, four-agent) examples demonstrate that CrewAI's architecture scales naturally from simple two-step pipelines to complex multi-stakeholder engineering workflows.

End of Week 3 — CrewAI: Autonomous Agents for Complex Tasks